

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL3 Konsep Dasar Pemrograman Berorientasi Objek



Azizatul Ainy
103132400030
S1DS REG 05-01

PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

a. Abstraksi

Menentukan hal-hal yang esensial pada suatu objek dan mengabaikan hal-hal yang sifatnya insidental. Seperti menentukan ciri-ciri (atribut) yang dimiliki oleh objek tersebut dan apa saja yang dapat dilakukan (metode) oleh objek.

b. Enkapsulasi

Proses pemaketan data objek bersama method-methodnya. Manfaat utama pengkapsulan yaitu menyembunyikan rincian implementasi dari objek lain yang tidak berhak.

Fungsi dari enkapsulasi:

i. Penyembunyian data

Objek disusun dari antarmuka *public Method* dan *private* data. Manfaat utama adalah bagian internal dapat berubah tanpa mempengaruhi bagian-bagian program lain.

ii. Modularitas

Kode sumber bagian internal objek dikelola secara terpisah dari antarmuka, maka bebas melakukan modifikasi yang tidak menyebabkan masalah pada bagian-bagian lain.

c. Inheritance

Proses penciptaan kelas baru dengan mewarisi karakteristik dari kelas yang telah ada, ditambah karakteristik unik kelas baru itu. Jika superclass telah mendefinisikan perilaku yang diperlukan, maka tidak perlu mendefinisikan ulang perilaku itu, cukup membuat kelas yang merupakan *subclass* dari *superclass* yang dimaksud.

d. Polymorphisms

Konsep ini memungkinkan digunakan antarmuka yang sama untuk memerintah suatu objek agar dapat melakukan aksi yang mungkin secara prinsip sama tapi secara proses berbeda. Mekanisme *polymorphism* dapat dilakukan dengan beberapa cara, seperti *Overloading Method*, *Overloading Constructor*, maupun *Overring Method*.

e. Kelas

Kelas dapat mendeklarasikan variabel dan menciptakan *Object*. Sebuah kelas mempunyai anggota yang terdiri atas atribut dan method. Dikatakan fungsi bila *method* tersebut melakukan suatu proses dan mengembalikan suatu nilai (return value), dan dikatakan prosedur bila *method* tersebut hanya melakukan suatu proses dan tidak mengembalikan nilai (void).

```
class <classname>
```

```
{
```

```
//declaration of data member
```

```
//declaration of Methods
```

```
}
```

f. Object

Insatansiasi atau hasil ciptaan dari suatu kelas yang telah dibuat sebelumnya. Dalam pengembangan program orientasi objek lebih lanjut, sebuah objek dimungkinkan terdiri atas objek-objek lain.

g. Field

Dapat berupa variabel kelas, variabel objek, variabel method objek atau parameter dari sebuah fungsi. Field merupakan anggota dari kelas yang digunakan untuk menyimpan data. Terdapat dua field dalam kelas;

i. Class field

Field yang dikaitkan dengan kelas dimana dia didefinisikan, bukan dengan sebuah *instance* dari kelas.

ii. Instance field

Field apapun yang dideklarasikan tanpa menggunakan *static modifier*.

h. Method

Dalam OOP, Method digunakan untuk memodularisasi program melalui pemisahan tugas dalam suatu kelas. Pemanggilan Method menspesifikasikan nama Method dan menyediakan informasi (parameter) yang diperlukan untuk melaksanakan tugasnya. Deklarasi Method untuk yang mengembalikan nilai (fungsi);

```
[modifier] Type-data namaMethod(parameter1,parameter2,...parameterN)
```

```
{
```

```
    Deklarasi-deklarasi dan proses ;
```

```
    return nilai-kembalian
```

```
}
```

Deklarasi Method untuk yang tidak mengembalikan nilai (prosedur).

```
[[modifier] void namaMethod(parameter1,parameter2,...parameterN)
{
    Deklarasi-deklarasi dan proses ;
}
```

Terdapat dua cara melewatkan argumen ke Method, yaitu:

i. Melewatkan secara nilai (Pass by Value)

Digunakan untuk argumen yang mempunyai tipe data primitif (Byte, short, int, long, float, double, char, dan boolean).

ii. Melewatkan secara referensi (Pass by Reference)

Digunakan pada array dan objek. Prosesnya isi memory pada variabel array dan objek merupakan penunjuk ke alamat memory yang mengandung data sesungguhnya yang akan dioperasikan.

a. Keyword “this”

Digunakan pada array dan objek. Prosesnya isi memory pada variabel array dan objek merupakan penunjuk ke alamat memory yang mengandung data sesungguhnya yang akan dioperasikan.

B. Guided

1. Abstraksi

```
package com.azizatulainy.modul3.abstraksi;

public class Kucing {
    String nama;
    String ras;
    void bersuara(){
        System.out.println(nama + ras + "meow");
    }
}
```

```
package com.azizatulainy.modul3.abstraksi;

class Abstraksi {
    public static void main (String[] args){
        Kucing aren = new Kucing ();

        aren.nama = "Empal Gentong";
        aren.ras = " Mujaerr";
        aren.bersuara();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Modul3 ---
Empal Gentong Mujaerrmeow
-----
BUILD SUCCESS
-----
Total time: 3.160 s
Finished at: 2026-10-05T22:15:14+07:00
-----
```

2. Enkapsulasi

```
package Guided.enkapsulasi;

public class Rekening {
    private int saldo = 0;

    public void tambahsaldo (int jumlah ){
        saldo = saldo + jumlah;
        System.out.println("Saldo berhasil ditambahkan");
    }

    public void tampilkansaldo(){
        System.out.println("Saldo anda " + saldo);
    }
}
```

```

    }
}

package Guided.enkapsulasi;

public class Rekening {
    private int saldo = 0;

    public void tambahsaldo (int jumlah ){
        saldo = saldo + jumlah;
        System.out.println("Saldo berhasil ditambahkan");
    }

    public void tampilkansaldo(){
        System.out.println("Saldo anda " + saldo);
    }
}

```

Output :

```

--- exec:3.6.3:exec (default-cli) @ Modul3 ---
Saldo berhasil ditambahkan
Saldo anda 2000000
Saldo berhasil ditambahkan
Saldo anda 17000000
-----
BUILD SUCCESS
-----
Total time:  1.816 s
Finished at: 2026-10-05T22:23:04+07:00
-----

```

3. Field

```

package Guidedfield;

public class Cirde {
    public static final double PI = 3.14159;

    public static double rediansToDegrees(double rads){
        return rads * 180 /PI;
    }

    public double r;

    public double area(){
        return PI * r * r;
    }

    public double circumference(){
        return 2 * PI * r;
    }
}

```

```
}
```

Output :

Tidak ada

C. Unguided

1. Soal Unguided 1

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di `main` sebelum diserahkan untuk diolah.

Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun — bukan sesuatu yang berbeda-beda tiap kali data dibuat — sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya.

Buatlah satu class yang bertugas mengolah data suhu ini — memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

Jawaban :

```
package Unguided;

public class PengolahSuhu {

    // Instance field: berbeda untuk setiap object, private agar tidak bisa diakses dari
    // luar
    private double[] suhuHarian;

    // Class field: milik class, dipakai bersama, nilainya tidak bisa diubah (final)
    private static final double NILAI_KOSONG = -1.0;

    // Nama parameter sama dengan nama field, sehingga 'this' dipakai secara
    // eksplisit.
    // Array TIDAK disalin; yang disimpan adalah referensi ke array yang sama.
    public PengolahSuhu(double[] suhuHarian) {
        this.suhuHarian = suhuHarian;
    }

    // Kebutuhan 3: tampilkan seluruh data, hari kosong ditandai jelas
    public void tampilkanData() {
        for (int i = 0; i < suhuHarian.length; i++) {
            if (suhuHarian[i] == NILAI_KOSONG) {
                System.out.println("Hari " + (i + 1) + " : (kosong)");
            } else {
                System.out.println("Hari " + (i + 1) + " : " + suhuHarian[i] + "°C");
            }
        }
    }
}
```

```

    }
}

// Kebutuhan 4: cari index hari kosong, -1 jika tidak ada
public int cariIndexKosong() {
    for (int i = 0; i < suhuHarian.length; i++) {
        if (suhuHarian[i] == NILAI_KOSONG) {
            return i;
        }
    }
    return -1;
}

// Kebutuhan 5: isi hari kosong dengan rata-rata hari sebelum dan sesudahnya
public void isiDataKosong() {
    int i = cariIndexKosong();
    if (i != -1) {
        suhuHarian[i] = (suhuHarian[i - 1] + suhuHarian[i + 1]) / 2;
    }
}

// Kebutuhan 6: rata-rata suhu dari data yang sudah bersih
public double hitungRataRata() {
    double total = 0;
    for (int i = 0; i < suhuHarian.length; i++) {
        total += suhuHarian[i];
    }
    return total / suhuHarian.length;
}
}

```

```

package Unguided;

import java.util.Arrays;

public class Mainsuhu {
    public static void main(String[] args) {
        // 1. Siapkan array suhu (Hari 1 = index 0)
        double[] suhuHarian = {30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9};

        // 2. Buat object, tampilkan data awal dan index kosong
        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

        System.out.println("=== Data Suhu Awal ===");
        pengolah.tampilkanData();
        System.out.println();
        System.out.println("Index hari kosong (dimulai dari 0): " +
            pengolah.cariIndexKosong());
        System.out.println();

        // 3. Isi data kosong
    }
}

```

```

        pengolah.isiDataKosong();

        // 4. Tampilkan data akhir dan rata-rata
        System.out.println("=== Data Suhu Setelah Pengisian ===");
        pengolah.tampilkanData();
        System.out.println();
        System.out.printf(java.util.Locale.US, "Rata-rata : %.2f°C%n",
        pengolah.hitungRataRata());
        System.out.println();

        // 5. Tampilkan array di main
        System.out.println("Isi array suhuHarian di main setelah isiDataKosong()
        dijalankan:");
        System.out.println(Arrays.toString(suhuHarian));
        System.out.println("(ikut berubah: constructor menyimpan referensi array
        yang sama)");

    }
}

```

Output :

```

--- exec:3.6.3:exec (default-cli) @ Modul3 ---
=== Data Suhu Awal ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : 29.1°C
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Rata-rata : 29.39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
(ikut berubah: constructor menyimpan referensi array yang sama)
-----
BUILD SUCCESS
-----
Total time: 1.975 s
Finished at: 2026-10-05T22:29:59+07:00
-----

```

Penjelasan :

Kode terdiri dari dua class. PengolahSuhu menyimpan array suhu (suhuHarian, dengan private) dan penanda data kosong -1.0 (NILAI_KOSONG, dengan static

final), lalu punya method untuk menampilkan data, mencari index yang kosong, mengisinya dengan rata-rata hari sebelum dan sesudahnya, dan menghitung rata-rata suhu. MainSuhu berisi main yang membuat array, membuat object PengolahSuhu, menjalankan semua method itu secara berurutan, dan menampilkan hasilnya, termasuk array di main yang ikut berubah karena array dikirim sebagai referensi, bukan salinan.

D. Kesimpulan

Praktikum Modul 3 membahas konsep dasar PBO dalam Java, yaitu abstraksi, enkapsulasi, class, object, field, method, serta *pass by value* dan *pass by reference*. Pada bagian guided, abstraksi diterapkan pada class `Kucing` yang hanya memuat atribut dan perilaku penting, enkapsulasi pada class `Rekening` yang menyembunyikan data `saldo` dengan `private`, dan field pada class `Cirde` yang membedakan *class field* (`PI`) dengan *instance field* (`r`). Pada bagian unguided, seluruh konsep tersebut digabungkan dalam program pengolah data suhu: class `PengolahSuhu` berhasil menemukan data kosong, mengisinya dengan rata-rata hari sebelum dan sesudahnya (29.1°C), dan menghitung rata-rata suhu seminggu (29.39°C). Array di `main` ikut berubah karena array dikirim sebagai referensi, bukan salinan. Dari praktikum ini dapat disimpulkan bahwa PBO membuat program lebih terstruktur, datanya lebih aman, dan kodenya lebih mudah digunakan kembali.